DOCUMENTACIÓN TÉCNICA PROFESIONAL

EJEMPLO

Arquitectura · API · Código · Calidad

25	55	4	11
ARCHIVOS	FUNCIONES	CLASES	ENDPOINTS

Lenguaje principal: Python
Score de calidad: 61/100 · Aceptable
Generado: 16/10/2026 22:05 UTC

Generado por AutoDocs AI

Tabla de Contenido

Tabla de Contenido 2

Documentación del Proyecto 5

Resumen Ejecutivo . . . 5

Stack Tecnológico . . . 5

Estadísticas del Proyecto . . . 5

2. Documentación de Arquitectura y Diseño . . . 6

2.1 Estructura de Directorios . . . 6

2.2 Distribución por Capas . . . 6

2.3 Distribución de Lenguajes . . . 6

2.4 Diagrama de Infraestructura . . . 7

2.5 Diagrama de Casos de Uso . . . 8

2.6 Diagrama de Clases . . . 8

2.7 Patrones Detectados . . . 9

2.8 Insights de Arquitectura (IA) . . . 9

3. Documentación Técnica de Código . . . 10

3.1 Guía de Configuración del Entorno de Desarrollo . . . 10

3.2 Catálogo de Endpoints de API . . . 10

3.3 Estado de Comentarios y Docstrings . . . 17

3.4 Escenarios de Pruebas . . . 19

4. Documentación de Módulos y Clases . . . 20

4.1 Tabla Resumen de Módulos . . . 20

4.2 Detalle de Clases . . . 20

1. Propósito de la clase (en palabras muy simples) . . . 20

2. Qué representa esta clase en el sistema (una analogía) . . . 20

3. Qué puede hacer esta clase (funcionalidades principales, explicadas de forma sencilla) . . 21

1. Propósito de la clase (en lenguaje simple) . . . 21

2. Qué representa en el sistema (analogía del mundo real) . . . 21

3. Qué puede hacer esta clase (funcionalidades principales) . . . 21

1. Propósito de la clase (¿para qué sirve?) . . . 22

2. Qué representa en el sistema (una analogía del mundo real) . . . 22

3. Qué puede hacer esta clase (funcionalidades principales) . . . 22

5. Documentación de Funciones . . . 23

5.1 Resumen por Módulo . . . 23

5.2 Detalle por Módulo . . . 23

6. Modelos de Datos . . . 27

Esquema General Inferido . . . 27

7. Guía de Despliegue y Mantenimiento . . . 27

7.1 Requisitos del Sistema . . . 27

7.2 Instalación Local . . . 27

7.3 Despliegue con Docker . . . 27

7.4 Variables de Entorno Recomendadas . . . 28

7.5 Checklist de Despliegue . . . 28

7.6 Registro de Cambios (Changelog) . . . 29

8. Reporte de Calidad del Código . . . 29

8.1 Análisis de Métricas . . . 29

8.2 Funciones con Alta Complejidad Ciclomática . . . 29

8.3 Recomendaciones . . . 29

8.4 Errores de Análisis 30

9. Análisis de Datos del Proyecto 30

9.1 Tipos y Fuentes de Datos Detectados 30

9.2 Esquema de Data Warehouse Inferido 30

9.3 Reporte de Calidad de Datos 30

9.4 Minería de Datos Aplicada al Código 31

9.5 Pipeline ETL Utilizado 34

Ejemplo

Documentación Técnica · AutoDocs AI · 16/10/2026

25	55	4	11
Archivos	Funciones	Clases	Endpoints

Score de Calidad: 61/100

Documentación del Proyecto

Generado automáticamente por AutoDocs AI — 2026-10-16 22:04 UTC

Resumen Ejecutivo

Este proyecto está desarrollado principalmente en **Python** y cuenta con 25 archivos fuente, 55 funciones, 4 clases y 11 endpoints de API documentados. El score de calidad del código es **61/100**.

Stack Tecnológico

Lenguaje	Cantidad
Python	16 archivos
HTML	5 archivos
JavaScript	3 archivos
CSS	1 archivos

Estadísticas del Proyecto

Métrica	Valor
Total de archivos	25
Funciones	55
Clases	4

Métrica	Valor
Endpoints API	11
Score de calidad	61/100
Funciones documentadas	12 / 55
Clases documentadas	0 / 4

---

2. Documentación de Arquitectura y Diseño

2.1 Estructura de Directorios

```
proyecto/
■■■ (raíz)/
■■■ controllers/
■■■ template/
■■■ services/
■■■ static/
■■■ models/
```

2.2 Distribución por Capas

• `(raíz)/` — 9 archivos [.py(5), .md(1), .txt(2), .gitignore(1)]

• `controllers/` — 6 archivos [.py(6)]

• `template/` — 5 archivos [.html(5)]

• `services/` — 1 archivos [.py(1)]

• `static/` — 5 archivos [.js(3), .css(1), .png(1)]

• `models/` — 4 archivos [.py(4)]

2.3 Distribución de Lenguajes

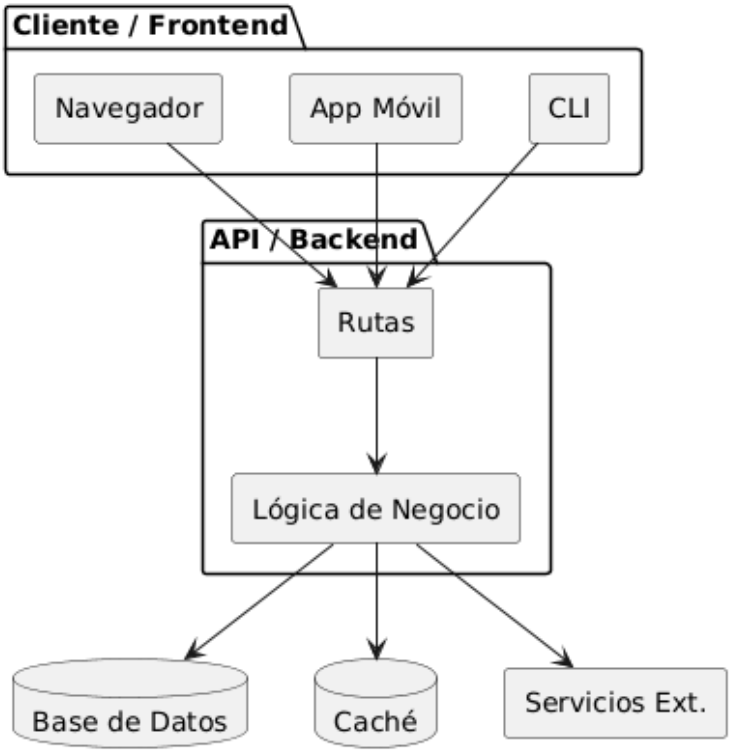
• Python: 16 archivos (64%)

• HTML: 5 archivos (20%)

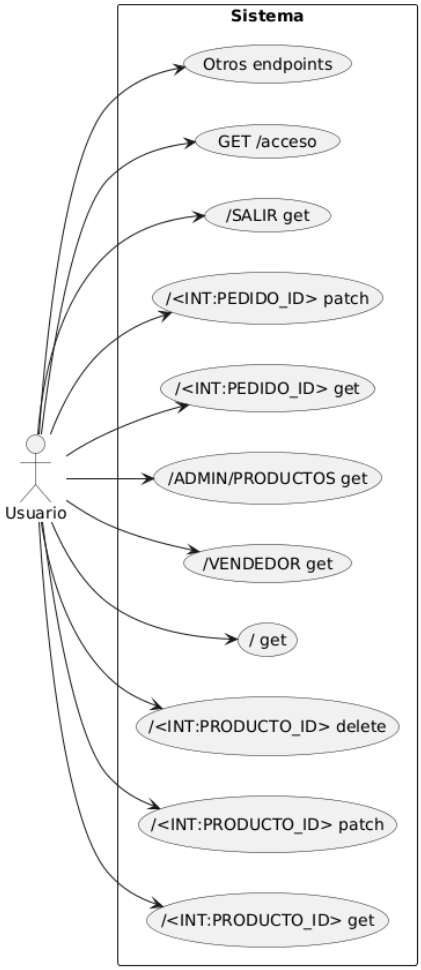
• JavaScript: 3 archivos (12%)

• CSS: 1 archivos (4%)

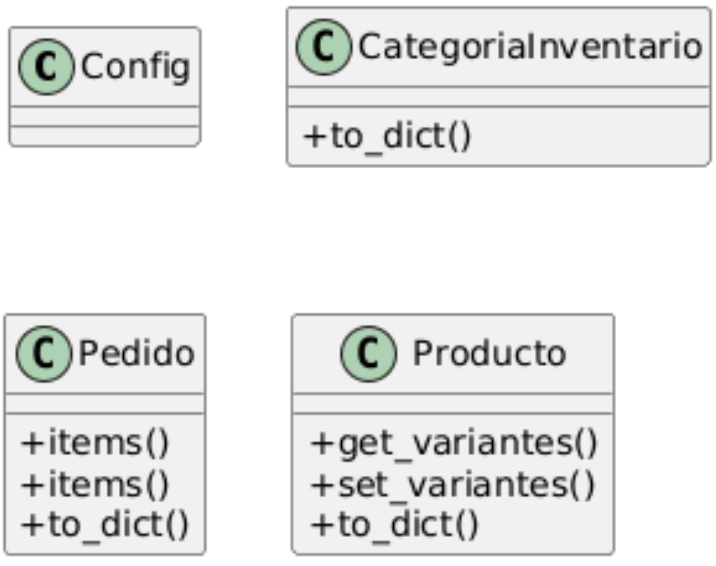
2.4 Diagrama de Infraestructura



2.5 Diagrama de Casos de Uso



2.6 Diagrama de Clases



■■ Clases con información incompleta: Las siguientes clases aparecen sin métodos ni atributos documentados: `Config`. Para que el diagrama se visualice correctamente, agrega métodos, propiedades o docstrings a estas clases en tu código fuente y regenera la documentación.

2.7 Patrones Detectados

• **API REST**: 11 endpoints detectados (Flask, Flask/FastAPI)

• **Programación Asíncrona**: 6 funciones async/await

2.8 Insights de Arquitectura (IA)

1. Patrones arquitectónicos probables

• Con 55 funciones, 4 clases y 11 endpoints, el proyecto parece un **monolito web** (probable Flask/Django) que sigue una variante ligera de **MVC**: vistas (endpoints), controladores (funciones) y modelos (clases).

• La presencia de varios lenguajes front■end (HTML, JS, CSS) indica una capa de presentación servida por el mismo proceso, no microservicios.

2. Buenas prácticas detectadas

• Uso de clases para encapsular lógica de dominio (aunque solo 4).

• Separación básica de responsabilidades: endpoints claros y funciones reutilizables.

• Código mayormente Python (16/25 archivos), lo que favorece la mantenibilidad.

3. Áreas de mejora sugeridas

• **Modularización**: agrupar funciones relacionadas en paquetes o módulos; evitar “funciones sueltas” en el nivel raíz.

• **Capas de servicio**: extraer la lógica de negocio a una capa de servicios/uso■cases para que los controladores queden delgados.

• **Validación y manejo de errores**: centralizar validaciones y respuestas HTTP con middleware o decoradores.

• **Testing**: añadir pruebas unitarias y de integración (actualmente no se menciona cobertura).

4. Complejidad del proyecto

• Funciones por clase ≈ 13, lo que sugiere clases con alta responsabilidad.

• 11 endpoints sobre 55 funciones → **densidad alta** de lógica en los controladores, lo que eleva la complejidad cognitiva.

5. Recomendaciones de escalabilidad

• **Refactorizar a

3. Documentación Técnica de Código

3.1 Guía de Configuración del Entorno de Desarrollo

```
# 1. Clonar el repositorio
git clone <url-del-repositorio>
cd <nombre-del-proyecto>

# 2. Crear entorno virtual (recomendado para Python)
python -m venv venv
source venv/bin/activate  # Linux/Mac
venv\Scripts\activate     # Windows

# 3. Instalar dependencias
pip install -r requirements.txt

# 4. Configurar variables de entorno
cp .env.example .env
# Editar .env con tus valores locales

# 5. Ejecutar en modo desarrollo
python app.py  # o npm run dev
```

3.2 Catálogo de Endpoints de API

El proyecto expone **11 endpoints**. A continuación se presenta el catálogo completo organizado por módulo:

Método	Ruta	Archivo	Framework
/<INT:PRODUCTO_ID>	`get`	`controllers/producto_controller.py`	Flask/FastAPI
/<INT:PRODUCTO_ID>	`patch`	`controllers/producto_controller.py`	Flask/FastAPI
/<INT:PRODUCTO_ID>	`delete`	`controllers/producto_controller.py`	Flask/FastAPI
/	`get`	`controllers/paginas_controller.py`	Flask/FastAPI
/VENDEDOR	`get`	`controllers/paginas_controller.py`	Flask/FastAPI
/ADMIN/PRODUCTOS	`get`	`controllers/paginas_controller.py`	Flask/FastAPI
/<INT:PEDIDO_ID>	`get`	`controllers/pedido_controller.py`	Flask/FastAPI
/<INT:PEDIDO_ID>	`patch`	`controllers/pedido_controller.py`	Flask/FastAPI
/SALIR	`get`	`controllers/auth_controller.py`	Flask/FastAPI

Método	Ruta	Archivo	Framework
GET	`/acceso`	`controllers/auth_controller.py`	Flask
POST	`/acceso`	`controllers/auth_controller.py`	Flask

Módulo: `controllers/producto_controller.py`

`/<INT:PRODUCTO_ID> get`

1. Propósito del endpoint (¿para qué sirve?)

Este enlace del programa permite obtener, de forma rápida y sencilla, los datos de un producto concreto que está guardado en la base de datos. Es como preguntar "¿qué información tiene este artículo?" y recibir la respuesta.

2. Qué hace este endpoint (funcionalidad)

Cuando alguien llama a la URL `/123` (donde *123* es el número que identifica al producto), el servidor busca ese producto en su registro y le devuelve los datos que tiene guardados: nombre, descripción, precio, disponibilidad, etc. Si el número no corresponde a ningún producto, responde diciendo que no lo encontró.

3. Qué información necesita para funcionar (qué se debe proporcionar)

Solo se necesita un dato: el **identificador numérico** del producto (un número entero). Ese número se escribe directamente en la dirección de la petición, por ejemplo:

```
GET https://mi■tienda.com/45
```

En este caso, "45" es el **PRODUCTO_ID** que el endpoint necesita para buscar el artículo correcto.

4. Qué resultado devuelve (respuesta)

• **Si el producto existe**: devuelve un pequeño paquete de información en formato fácil de leer (JSON). Contendrá campos

Propiedad	Valor
Framework	Flask/FastAPI
Archivo	`controllers/producto_controller.py`
Auth	Bearer JWT Token
Código	Descripción
---	---
`200`	Operación exitosa

Propiedad	Valor
`400`	Solicitud incorrecta
`401`	No autorizado
`404`	Recurso no encontrado
`500`	Error interno del servidor

`/<INT:PRODUCTO_ID> patch`

1. Propósito del endpoint (qué quiere conseguir)

Este punto de la aplicación permite **actualizar los datos de un producto que ya está registrado**. Imagina que tienes una hoja de cálculo con la lista de productos de tu tienda; si cambias el precio, la descripción o la disponibilidad de uno de ellos, este endpoint es el que guarda esos cambios en el sistema.

---

2. Qué hace este endpoint (funcionalidad)

• Recibe la identificación (número) del producto que se quiere modificar.

• Lee la información nueva que le envías (por ejemplo, “el precio ahora es 12,99” o “el producto está agotado”).

• Busca ese producto en la base de datos y **reemplaza** los datos viejos por los nuevos.

• Devuelve una confirmación de que la actualización se realizó correctamente o un mensaje de error si algo no salió bien (por ejemplo, si el número de producto no existe).

En resumen: **“Encuentra el producto indicado y le cambia los datos que le indiques”.**

---

3. Qué información necesita para funcionar

| Dato que se envía | Qué significa | Ejemplo |

|---|---|---|

| `PRODUCTO_ID` (en la URL) | El

Propiedad	Valor
Framework	Flask/FastAPI
Archivo	`controllers/producto_controller.py`
Auth	Bearer JWT Token

Propiedad	Valor
Código	Descripción
---	---
`200`	Operación exitosa
`400`	Solicitud incorrecta
`401`	No autorizado
`404`	Recurso no encontrado
`500`	Error interno del servidor

`/<INT:PRODUCTO_ID> delete`

1. Propósito del endpoint (¿para qué sirve?)

Este recurso permite **eliminar un producto** del catálogo del sistema. Es como "borrar" un artículo que ya no está disponible o que se ingresó por error.

---

2. Qué hace este endpoint (funcionalidad)

Cuando se llama a esta URL con el método **DELETE**, el servidor busca el producto cuyo número de identificación (ID) se indica en la dirección y, si lo encuentra, lo quita de la base de datos. Después de la eliminación, el sistema confirma que la operación se realizó.

---

3. Qué información necesita para funcionar

• **El ID del producto** que se quiere borrar.

- Ese ID es un número entero y se escribe justo después de la barra en la URL, por ejemplo: `DELETE /12345`.

• No se envía ningún otro dato ni cuerpo de mensaje; solo el número que identifica al producto.

---

4. Qué resultado devuelve

• **Éxito**: un mensaje sencillo que indica que el producto fue eliminado, normalmente algo como

```
{ "mensaje": "Producto eliminado correctamente" }
```

(con un código de respuesta

Propiedad	Valor
Framework	Flask/FastAPI
Archivo	`controllers/producto_controller.py`
Auth	Bearer JWT Token
Código	Descripción
---	---
`200`	Operación exitosa
`400`	Solicitud incorrecta
`401`	No autorizado
`404`	Recurso no encontrado
`500`	Error interno del servidor

Módulo: `controllers/paginas_controller.py`

`/` `get`

Propiedad	Valor
Framework	Flask/FastAPI
Archivo	`controllers/paginas_controller.py`
Auth	Bearer JWT Token
Código	Descripción
---	---
`200`	Operación exitosa
`400`	Solicitud incorrecta
`401`	No autorizado
`404`	Recurso no encontrado
`500`	Error interno del servidor

`/VENDEDOR` `get`

Propiedad	Valor
Framework	Flask/FastAPI
Archivo	`controllers/paginas_controller.py`
Auth	Bearer JWT Token
Código	Descripción

Propiedad	Valor
---	---
`200`	Operación exitosa
`400`	Solicitud incorrecta
`401`	No autorizado
`404`	Recurso no encontrado
`500`	Error interno del servidor

`/ADMIN/PRODUCTOS get`

Propiedad	Valor
Framework	Flask/FastAPI
Archivo	`controllers/paginas_controller.py`
Auth	Bearer JWT Token
Código	Descripción
---	---
`200`	Operación exitosa
`400`	Solicitud incorrecta
`401`	No autorizado
`404`	Recurso no encontrado
`500`	Error interno del servidor

Módulo: `controllers/pedido_controller.py`

`/<INT:PEDIDO_ID> get`

Propiedad	Valor
Framework	Flask/FastAPI
Archivo	`controllers/pedido_controller.py`
Auth	Bearer JWT Token
Código	Descripción
---	---
`200`	Operación exitosa
`400`	Solicitud incorrecta
`401`	No autorizado

Propiedad	Valor
`404`	Recurso no encontrado
`500`	Error interno del servidor

`/<INT:PEDIDO_ID> patch`

Propiedad	Valor
Framework	Flask/FastAPI
Archivo	`controllers/pedido_controller.py`
Auth	Bearer JWT Token
Código	Descripción
---	---
`200`	Operación exitosa
`400`	Solicitud incorrecta
`401`	No autorizado
`404`	Recurso no encontrado
`500`	Error interno del servidor

Módulo: `controllers/auth_controller.py`

`/SALIR get`

Propiedad	Valor
Framework	Flask/FastAPI
Archivo	`controllers/auth_controller.py`
Auth	Bearer JWT Token
Código	Descripción
---	---
`200`	Operación exitosa
`400`	Solicitud incorrecta
`401`	No autorizado
`404`	Recurso no encontrado
`500`	Error interno del servidor

`GET /acceso`

Propiedad	Valor
Framework	Flask
Archivo	`controllers/auth_controller.py`
Auth	Bearer JWT Token
Código	Descripción
---	---
`200`	Operación exitosa
`400`	Solicitud incorrecta
`401`	No autorizado
`404`	Recurso no encontrado
`500`	Error interno del servidor

`POST /acceso`

Propiedad	Valor
Framework	Flask
Archivo	`controllers/auth_controller.py`
Body	JSON (requerido)
Auth	Bearer JWT Token
Código	Descripción
---	---
`200`	Operación exitosa
`400`	Solicitud incorrecta
`401`	No autorizado
`404`	Recurso no encontrado
`500`	Error interno del servidor

3.3 Estado de Comentarios y Docstrings

Métrica	Valor
Total de funciones	55
Documentadas	12 (22%)
Sin documentar	43 (78%)

```
[■■■■■■■■■■■■■■■■■■■■■] 22% documentado
```

Fragmentos de Código con Documentación

A continuación se muestran fragmentos reales del proyecto que cuentan con docstrings correctamente implementados:

```
def _proteger_productos_admin():
    """
    Todo este blueprint es solo para el vendedor autenticado.
    """
    ...
```

```
def listar_productos_admin():
    """
    Lista todos los productos con inventario de categoría visible par
a el vendedor
    """
    ...
```

```
def crear_producto():
    """
    Crea un nuevo producto
    """
    ...
```

```
def obtener_producto(producto_id):
    """
    Obtiene un producto específico con inventario de categoría
    """
    ...
```

Funciones que Requieren Documentación

Función	Parámetros	Archivo
`create_app`	—	`app.py`
`index`	—	`controllers/paginas_controller.py`
`vendedor`	—	`controllers/paginas_controller.py`
`admin_productos`	—	`controllers/paginas_controller.py`
`crear_pedido`	—	`controllers/pedido_controller.py`
`listar_pedidos`	—	`controllers/pedido_controller.py`
`obtener_pedido`	`pedido_id`	`controllers/pedido_controller.py`
`actualizar_pedido`	`pedido_id`	`controllers/pedido_controller.py`

Función	Parámetros	Archivo
`acceso`	—	`controllers/auth_controller.py`
`salir`	—	`controllers/auth_controller.py`

... y 33 funciones más sin documentar.

3.4 Escenarios de Pruebas

■■ No se detectaron pruebas automatizadas en este proyecto.

Se recomienda implementar los siguientes escenarios de prueba:

Endpoint	Escenario	Tipo	Prioridad
`/<INT:PRODUCTO_ID> get`	Respuesta exitosa (200)	Integración	Alta
`/<INT:PRODUCTO_ID> get`	Sin autenticación (401)	Seguridad	Alta
`/<INT:PRODUCTO_ID> patch`	Respuesta exitosa (200)	Integración	Alta
`/<INT:PRODUCTO_ID> patch`	Sin autenticación (401)	Seguridad	Alta
`/<INT:PRODUCTO_ID> delete`	Respuesta exitosa (200)	Integración	Alta
`/<INT:PRODUCTO_ID> delete`	Sin autenticación (401)	Seguridad	Alta
`/ get`	Respuesta exitosa (200)	Integración	Alta
`/ get`	Sin autenticación (401)	Seguridad	Alta
`/VENDEDOR get`	Respuesta exitosa (200)	Integración	Alta
`/VENDEDOR get`	Sin autenticación (401)	Seguridad	Alta
`/ADMIN/PRODUCTOS get`	Respuesta exitosa (200)	Integración	Alta
`/ADMIN/PRODUCTOS get`	Sin autenticación (401)	Seguridad	Alta
`/<INT:PEDIDO_ID> get`	Respuesta exitosa (200)	Integración	Alta
`/<INT:PEDIDO_ID> get`	Sin autenticación (401)	Seguridad	Alta
`/<INT:PEDIDO_ID> patch`	Respuesta exitosa (200)	Integración	Alta
`/<INT:PEDIDO_ID> patch`	Sin autenticación (401)	Seguridad	Alta

Ejemplo de implementación recomendada:

```
import pytest

class TestEndpoints:
    def test_get_returns_200(self, client):
        response = client.get('/api/endpoint')
        assert response.status_code == 200

    def test_post_creates_resource(self, client):
```

```
        data = {'name': 'test'}
        response = client.post('/api/endpoint', json=data)
        assert response.status_code == 201

    def test_unauthorized_returns_401(self, client):
        response = client.get('/api/protected')
        assert response.status_code == 401
```

---

4. Documentación de Módulos y Clases

El proyecto contiene **4 clases** distribuidas en los siguientes módulos.

4.1 Tabla Resumen de Módulos

Clase	Archivo	Línea	Herencia	Métodos	Documentada
`Config`	`config.py`	8	object	0	No
`CategoriaInventario`	`models/categoria_inventario.py`	4	db.Model	1	No
`Pedido`	`models/pedido.py`	8	db.Model	3	No
`Producto`	`models/producto.py`	5	db.Model	3	No

4.2 Detalle de Clases

`Config`

Descripción sencilla de la clase `Config`

---

1. Propósito de la clase (en palabras muy simples)

`Config` es como una **caja de herramientas** donde se guardan los ajustes y valores que necesita el programa para funcionar.

En lugar de escribir esos valores una y otra vez en diferentes partes del código, la clase los mantiene organizados y fáciles de encontrar.

---

2. Qué representa esta clase en el sistema (una analogía)

Imagina que tienes una **cocina**.

• Los ingredientes (sal, azúcar, aceite) están siempre en la despensa.

• Cuando vas a cocinar, no vas a buscar cada ingrediente por toda la casa; simplemente vas a la despensa y tomas lo que necesitas.

`Config` es esa **despensa** del proyecto: contiene “ingredientes” (configuraciones, rutas de archivos, claves de acceso, valores por defecto, etc.) que el resto del programa utiliza cuando lo necesita.

---

3. Qué puede hacer esta clase (funcionalidades principales, explicadas de forma sencilla)

| Acción | Qué significa en la vida real | Cómo lo hace la clase `Config` |

|--------|-------------------------------|--------------------------------|

| **Guardar valores** | Anotar en una libreta los números de

`CategoriaInventario`

Descripción sencilla de la clase `CategoriaInventario`

---

1. Propósito de la clase (en lenguaje simple)

Imagina que tienes una tienda o un almacén y necesitas organizar los productos en grupos: “Bebidas”, “Papelería”, “Herramientas”, etc.

La clase `CategoriaInventario` es la “plantilla” que usamos en el programa para crear y manejar esos grupos de productos. Cada vez que el sistema necesita saber a qué categoría pertenece un artículo, recurre a esta clase.

---

2. Qué representa en el sistema (analogía del mundo real)

Analogía: Piensa en una hoja de cálculo con una columna llamada “Categoría”. Cada fila (un producto) tiene escrito en esa columna el nombre del grupo al que pertenece.

En el programa, la hoja de cálculo es la base de datos y la columna “Categoría” está representada por la clase `CategoriaInventario`. Cada instancia (cada registro) de la clase equivale a una fila de esa columna: “Bebidas”, “Limpieza”, “Electrónica”, etc.

---

3. Qué puede hacer esta clase (funcionalidades principales)

| Acción que realiza la clase

Métodos disponibles
`to_dict()`

`Pedido`

Clase `Pedido` – Explicación sencilla

---

1. Propósito de la clase (¿para qué sirve?)

Imagina que en una tienda necesitas registrar cada compra que hacen los clientes. La clase **`Pedido`** es la herramienta que el programa usa para crear y guardar esa "hoja de pedido" dentro de la base de datos. Cada vez que alguien pide uno o varios productos, se crea un objeto **`Pedido`** que contiene toda la información del pedido.

---

2. Qué representa en el sistema (una analogía del mundo real)

Pedido = Recibo o factura de una compra

En la vida real, cuando vas al supermercado y te entregan un ticket con los productos que llevas, la cantidad y el precio, ese ticket es el "pedido". En la aplicación, la clase **`Pedido`** es ese ticket digital: guarda quién compró, cuándo lo hizo y qué artículos forman parte de la compra.

---

3. Qué puede hacer esta clase (funcionalidades principales)

| Acción | Qué hace realmente la clase | Explicación fácil |

|--------|----------------------------|-------------------|

| **`items`** (propiedad) | Devuelve la lista de productos que forman parte del

Métodos disponibles
`items()`
`items()`
`to_dict()`

`Producto`

Métodos disponibles
`get_variantes()`
`set_variantes()`
`to_dict()`

5. Documentación de Funciones

El proyecto contiene **55 funciones** distribuidas en **13 módulos**.

5.1 Resumen por Módulo

Módulo	Funciones	Complejidad Prom.	Documentadas	Async
`app.py`	1	2.0	0%	—
`controllers/producto_controller.py`	6	6.3	100%	—
`controllers/paginas_controller.py`	3	1.0	0%	—
`controllers/menu_controller.py`	1	2.0	100%	—
`controllers/pedido_controller.py`	5	6.6	20%	—
`controllers/auth_controller.py`	4	3.0	25%	—
`services/whatsapp_service.py`	3	4.3	100%	—
`static/js/cart.js`	11	1.0	0%	1
`static/js/vendedor.js`	5	1.0	0%	3
`static/js/productos_admin.js`	9	1.0	0%	2
`models/categoria_inventario.py`	1	1.0	0%	—
`models/pedido.py`	3	1.0	0%	—
`models/producto.py`	3	2.3	0%	—

5.2 Detalle por Módulo

Módulo: `app.py`

Función	Parámetros	Líneas de código	Complejidad	Doc
`create_app`	–	—	2	

Módulo: `controllers/producto_controller.py`

Función	Parámetros	Líneas de código	Complejidad	Doc
`_proteger_productos_admin`	–	—	1	
`listar_productos_admin`	–	—	3	
`crear_producto`	–	—	12	
`obtener_producto`	`producto_id`	—	1	
`actualizar_producto`	`producto_id`	—	20	
`eliminar_producto`	`producto_id`	—	1	

Módulo: `controllers/paginas_controller.py`

Función	Parámetros	Líneas de código	Complejidad	Doc
`index`	–	—	1	
`vendedor`	–	—	1	
`admin_productos`	–	—	1	

Módulo: `controllers/menu_controller.py`

Función	Parámetros	Líneas de código	Complejidad	Doc
`listar_productos`	–	—	2	

Módulo: `controllers/pedido_controller.py`

Función	Parámetros	Líneas de código	Complejidad	Doc
`_proteger_pedidos_vendedor`	–	—	2	
`crear_pedido`	–	—	22	
`listar_pedidos`	–	—	3	
`obtener_pedido`	`pedido_id`	—	1	
`actualizar_pedido`	`pedido_id`	—	5	

Módulo: `controllers/auth_controller.py`

Función	Parámetros	Líneas de código	Complejidad	Doc
`requiere_acceso`	`f`	—	3	
`acceso`	–	—	5	

Función	Parámetros	Líneas de código	Complejidad	Doc
`salir`	—	—	1	
`decorador`	—	—	3	

Módulo: `services/whatsapp_service.py`

Función	Parámetros	Líneas de código	Complejidad	Doc
`_limpiar_numero`	`numero`	—	3	
`link_pedido_para_vendedor`	`numero_vendedor, pedido`	—	7	
`link_aviso_para_cliente`	`pedido`	—	3	

Módulo: `static/js/cart.js`

Función	Parámetros	Líneas de código	Complejidad	Doc
`cargarMenu`	—	—	1	
`productoCardHTML`	`p`	—	1	
`agregarAlCarrito`	`producto`	—	1	
`cambiarCantidad`	`id, delta, variantesEncoded = ""`	—	1	
`renderCarrito`	—	—	1	
`abrirTicket`	—	—	1	
`cerrarTicket`	—	—	1	
`abrirModalVariantes`	`producto, seleccionActual = {}`	—	1	
`editarItemDelCarrito`	`id, variantesEncoded`	—	1	
`cerrarModalVariantes`	—	—	1	
`mostrarError`	`msg`	—	1	

Módulo: `static/js/vendedor.js`

Función	Parámetros	Líneas de código	Complejidad	Doc
`renderFiltros`	—	—	1	
`cargarPedidos`	—	—	1	
`pedidoCardHTML`	`p`	—	1	

Función	Parámetros	Líneas de código	Complejidad	Doc
`guardarCambios`	`id`	—	1	
`avisarCliente`	`id`	—	1	

Módulo: `static/js/productos_admin.js`

Función	Parámetros	Líneas de código	Complejidad	Doc
`cargarProductos`	`—`	—	1	
`renderProductos`	`—`	—	1	
`productoCardHTML`	`p`	—	1	
`abrirModal`	`titulo = "Nuevo Producto"`	—	1	
`cerrarModal`	`—`	—	1	
`agregarGrupoVariante`	`nombre = "", opciones = ""`	—	1	
`eliminarVariante`	`btn`	—	1	
`editarProducto`	`id`	—	1	
`eliminarProducto`	`id`	—	1	

Módulo: `models/categoria_inventario.py`

Función	Parámetros	Líneas de código	Complejidad	Doc
`to_dict`	`self`	—	1	

Módulo: `models/pedido.py`

Función	Parámetros	Líneas de código	Complejidad	Doc
`items`	`self`	—	1	
`items`	`self, value`	—	1	
`to_dict`	`self`	—	1	

Módulo: `models/producto.py`

Función	Parámetros	Líneas de código	Complejidad	Doc
`get_variantes`	`self`	—	4	
`set_variantes`	`self, variantes_dict`	—	1	

Función	Parámetros	Líneas de código	Complejidad	Doc
`to_dict`	`self, incluir_inventario`	—	2	

6. Modelos de Datos

No se detectaron clases con nombres de modelo explícitos.

Esquema General Inferido

Basado en el análisis del código, las entidades principales son:

- **App**
- **Variantes**

7. Guía de Despliegue y Mantenimiento

7.1 Requisitos del Sistema

- **Lenguaje principal**: Python
- **Archivos detectados**: 25

7.2 Instalación Local

```
# 1. Clonar el repositorio
git clone <url-del-repositorio>
cd <nombre-del-proyecto>

# 2. Instalar dependencias
pip install -r requirements.txt

# 3. Configurar variables de entorno
cp .env.example .env
# Editar .env con tus valores

# 4. Ejecutar el proyecto
python app.py
```

7.3 Despliegue con Docker

```
FROM python:3.12-slim

WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
EXPOSE 8000
CMD ["python app.py"]
```

```
# Construir imagen
docker build -t mi-proyecto .

# Ejecutar contenedor
docker run -p 8000:8000 mi-proyecto
```

7.4 Variables de Entorno Recomendadas

```
# General
APP_ENV=production
SECRET_KEY=clave-secreta-segura
DEBUG=False

# Base de datos
DB_HOST=localhost
DB_PORT=5432
DB_NAME=mi_base_datos
DB_USER=usuario
DB_PASSWORD=contraseña

# API
API_VERSION=v1
ALLOWED_HOSTS=*
```

7.5 Checklist de Despliegue

- [ ] Variables de entorno configuradas
- [ ] Base de datos migrada
- [ ] Tests ejecutados y pasando
- [ ] Logs configurados
- [ ] HTTPS habilitado en producción
- [ ] Backups programados
- [ ] Monitoreo configurado

7.6 Registro de Cambios (Changelog)

Versión 1.0.0 — 2026-10-16

• Versión inicial analizada por AutoDocs AI

• Documentación técnica generada automáticamente

• Análisis de calidad del código completado

---

8. Reporte de Calidad del Código

8.1 Análisis de Métricas

Score de Calidad: 61/100 — **Bueno**

Distribución de Documentación:

• Funciones documentadas: 12/55 (22%)

• Clases documentadas: 0/4

Análisis de Complejidad:

• Complejidad promedio: 2.53

• Complejidad máxima: 22

• Funciones con alta complejidad (>10): 3

Tendencia de Complejidad por Archivo:

```
[Gráfico de regresión lineal mostrando la tendencia de complejidad]
Eje X: Archivos ordenados por complejidad promedio
Eje Y: Score de complejidad
Tendencia: → Estable
```

8.2 Funciones con Alta Complejidad Ciclomática

• `crear_producto` en `controllers/producto_controller.py` — complejidad: **12**

• `actualizar_producto` en `controllers/producto_controller.py` — complejidad: **20**

• `crear_pedido` en `controllers/pedido_controller.py` — complejidad: **22**

8.3 Recomendaciones

• Agregar docstrings a las 43 funciones sin documentar

• Refactorizar funciones con alta complejidad: `crear_producto`, `actualizar_producto`, `crear_pedido`

8.4 Errores de Análisis

Sin errores detectados. [OK]

---

9. Análisis de Datos del Proyecto

9.1 Tipos y Fuentes de Datos Detectados

Datos Estructurados: 0 archivos

Datos Semi-estructurados: 0 archivos

Datos No Estructurados: 0 archivos

9.2 Esquema de Data Warehouse Inferido

No se detectaron suficientes modelos de datos para generar un esquema de Data Warehouse.

9.3 Reporte de Calidad de Datos

Problemas Detectados en el Código:

• **Funciones sin documentar:** 43

- `create_app` en `app.py`
- `index` en `controllers/paginas_controller.py`
- `vendedor` en `controllers/paginas_controller.py`
- `admin_productos` en `controllers/paginas_controller.py`
- `crear_pedido` en `controllers/pedido_controller.py`
- ... y 38 más

• **Funciones de alta complejidad:** 3

- `crear_producto` en `controllers/producto_controller.py` (complejidad: 12)
- `actualizar_producto` en `controllers/producto_controller.py` (complejidad: 20)
- `crear_pedido` en `controllers/pedido_controller.py` (complejidad: 22)

9.4 Minería de Datos Aplicada al Código

Análisis automático realizado sobre las funciones y clases extraídas del proyecto.

A) Funciones de Alta Complejidad Ciclomática

Nivel	Umbral	Cantidad	Descripción
Alta	> 5	4	Requieren refactorización urgente
Media	4–5	3	Candidatas a simplificación
Baja	≤ 3	48	Complejidad aceptable

Funciones de alta complejidad — detalle y recomendaciones:

• **`crear_pedido [sin docstring]`**

- ■ Archivo: **`controllers/pedido_controller.py`** — línea 20

- ■ Complejidad ciclomática: **22**

- ■ Sugerencia: Complejidad crítica (22). Divide esta función en al menos 4 funciones más pequeñas con responsabilidad única. Cada rama lógica (if/for/while) debería ser su propio método.

• **`actualizar_producto`**

- ■ Archivo: **`controllers/producto_controller.py`** — línea 87

- ■ Complejidad ciclomática: **20**

- ■ Sugerencia: Complejidad crítica (20). Divide esta función en al menos 4 funciones más pequeñas con responsabilidad única. Cada rama lógica (if/for/while) debería ser su propio método.

• **`crear_producto`**

- ■ Archivo: **`controllers/producto_controller.py`** — línea 36

- ■ Complejidad ciclomática: **12**

- ■ Sugerencia: Complejidad alta (12). Extrae las ramas condicionales a funciones auxiliares privadas y considera el patrón de «guardas tempranas» (early return) para reducir el anidamiento.

• **`link_pedido_para_vendedor`**

- ■ Archivo: **`services/whatsapp_service.py`** — línea 9

- ■ Complejidad ciclomática: **7**

- ■ Sugerencia: Complejidad moderada-alta (7). Revisa si alguno de los bloques if/for puede extraerse a una función con nombre descriptivo.

B) Funciones Sin Documentación

43 de 55 funciones carecen de docstring (78%).

Función	Archivo	Línea	Complejidad
`create_app`	`app.py`	7	2
`index`	`controllers/paginas_controller.py`	8	1
`vendedor`	`controllers/paginas_controller.py`	16	1
`admin_productos`	`controllers/paginas_controller.py`	24	1
`crear_pedido`	`controllers/pedido_controller.py`	20	22
`listar_pedidos`	`controllers/pedido_controller.py`	91	3
`obtener_pedido`	`controllers/pedido_controller.py`	101	1
`actualizar_pedido`	`controllers/pedido_controller.py`	107	5
`acceso`	`controllers/auth_controller.py`	25	5
`salir`	`controllers/auth_controller.py`	50	1
`decorador`	`controllers/auth_controller.py`	14	3
`cargarMenu`	`static/js/cart.js`	?	1
`productoCardHTML`	`static/js/cart.js`	?	1
`agregarAlCarrito`	`static/js/cart.js`	?	1
`cambiarCantidad`	`static/js/cart.js`	?	1
`renderCarrito`	`static/js/cart.js`	?	1
`abrirTicket`	`static/js/cart.js`	?	1
`cerrarTicket`	`static/js/cart.js`	?	1
`abrirModalVariantes`	`static/js/cart.js`	?	1
`editarItemDelCarrito`	`static/js/cart.js`	?	1

... y 23 funciones más sin documentar.

■ **Sugerencia:** Agrega docstrings a todas las funciones públicas. Prioriza las de mayor complejidad primero, ya que son las más difíciles de entender sin documentación.

C) Detección de Funciones con Nombres Similares (Posibles Duplicados)

Se comparan los nombres de funciones entre archivos para detectar lógica duplicada o que podría unificarse.

Se detectaron **2 grupos de funciones con nombres similares** repartidas en distintos archivos:

Patrón: _to_dict / to_dict_ — aparece en 3 archivos distintos

Función	Archivo	Línea	Complejidad
`to_dict`	`models/categoria_inventario.py`	11	1
`to_dict`	`models/pedido.py`	33	1
`to_dict`	`models/producto.py`	29	2

■ **Sugerencia:** Las 3 funciones `to_dict`, `to_dict`, `to_dict` comparten el patrón `to_dict` en distintos módulos. Esto sugiere lógica duplicada. Extrae la lógica común a un servicio o función base y haz que cada módulo la llame con sus parámetros propios.

Patrón: _productocardhtml / productocardhtml_ — aparece en 2 archivos distintos

Función	Archivo	Línea	Complejidad
`productoCardHTML`	`static/js/cart.js`	?	1
`productoCardHTML`	`static/js/productos_admin.js`	?	1

■ **Sugerencia:** Las funciones `productoCardHTML` y `productoCardHTML` realizan operaciones similares (`productocardhtml`) en módulos distintos. Considera crear una función compartida en un módulo utilitario (por ejemplo `utils/productocardhtml.py` o `helpers/common.py`) que reciba el contexto como parámetro, y llamarla desde ambos archivos (`static/js/cart.js`, `static/js/productos_admin.js`).

D) Análisis de Módulos — Archivos con Mayor Concentración de Funciones

Archivo	Funciones	Complejidad Promedio	Documentadas	Estado
`static/js/cart.js`	11	1.0	0%	Revisar
`static/js/productos_admin.js`	9	1.0	0%	OK
`controllers/producto_controller.py`	6	6.3	100%	Revisar
`controllers/pedido_controller.py`	5	6.6	20%	Revisar
`static/js/vendedor.js`	5	1.0	0%	OK
`controllers/auth_controller.py`	4	3.0	25%	OK
`controllers/paginas_controller.py`	3	1.0	0%	OK
`services/whatsapp_service.py`	3	4.3	100%	OK

Archivo	Funciones	Complejidad Promedio	Documentadas	Estado
`models/pedido.py`	3	1.0	0%	OK
`models/producto.py`	3	2.3	0%	OK
`app.py`	1	2.0	0%	OK
`controllers/menu_controller.py`	1	2.0	100%	OK
`models/categoria_inventario.py`	1	1.0	0%	OK

■ `static/js/cart.js` tiene 11 funciones. Considera dividirlo en submódulos más específicos siguiendo el principio de responsabilidad única (SRP).

E) Patrones Frecuentes Adicionales (Análisis IA)

• **create**_: 1 ocurrencias

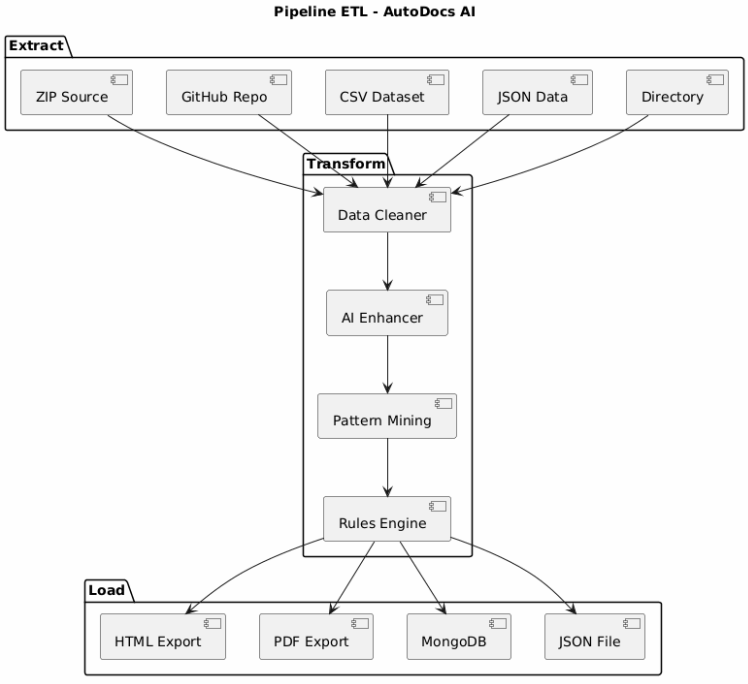
- Ejemplos: Operaciones create_ repetidas

• **get**_: 1 ocurrencias

- Ejemplos: Operaciones get_ repetidas

9.5 Pipeline ETL Utilizado

El proceso de generación de documentación sigue un pipeline ETL (Extract, Transform, Load):



Fases del Pipeline:

• **Extract:** Extracción de código desde repositorios GitHub, archivos ZIP o directorios locales

• **Transform:** Análisis del código, limpieza de datos, enriquecimiento con IA y minería de patrones

• **Load:** Generación de documentación en HTML, PDF o almacenamiento en MongoDB

Documentación generada automáticamente por AutoDocs AI — 2026-10-16 22:04 UTC

Generado automáticamente por AutoDocs AI